

4

Preparar y medir texto

Cómo limpiar, normalizar y tokenizar los tickets de Nube Sur antes de enviarlos a un modelo, y cómo medir con métricas y un registro si los resúmenes que genera son buenos.

APRENDIZAJE ESPERADO 4 · TEXTUAL DEL PLAN FORMATIVO

IMPLEMENTAR UN PIPELINE DE PREPROCESAMIENTO Y EVALUACIÓN BÁSICA DE TEXTO EN PYTHON, APLICANDO TÉCNICAS DE LIMPIEZA, NORMALIZACIÓN Y MÉTRICAS INICIALES DE CALIDAD, DE ACUERDO CON LOS REQUISITOS DE ENTRADA Y SALIDA DEL MODELO.

PLAN FORMATIVO

PF1822 · Especialización en Desarrollo con IA

MÓDULO

MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

CASO DE ESTUDIO

Nube Sur, empresa ficticia

TIEMPO DE LECTURA

20 minutos, más la autocomprobación

Qué vas a lograr

Al terminar esta lectura tendrás la base para demostrar los criterios de evaluación del aprendizaje esperado 4, textuales del plan formativo:

- 4.1** EXPLICA LOS CONCEPTOS FUNDAMENTALES DE NLP Y EVALUACIÓN BÁSICA DE RESPUESTAS, CONSIDERANDO TOKEN, LEMA, STOPWORDS, N-GRAMAS, COHERENCIA Y RELEVANCIA.

- 4.2** CODIFICA UNA RUTINA EN PYTHON PARA LA LIMPIEZA DE TEXTO, TOKENIZACIÓN Y NORMALIZACIÓN SOBRE UN CORPUS DE DOCUMENTOS, DE ACUERDO CON BUENAS PRÁCTICAS.

- 4.3** CALCULA MÉTRICAS BÁSICAS DE CALIDAD DE RESPUESTAS GENERADAS, CONSIDERANDO COHERENCIA, RELEVANCIA, BLEU Y ROUGE, REGISTRANDO LOS RESULTADOS OBTENIDOS DE ACUERDO CON UN PROTOCOLO DEFINIDO.

Cómo está organizada

1 Conceptos básicos de NLP	6 Vectorización básica
2 Limpieza	7 Un pipeline modular
3 Normalización	8 Métricas de calidad
4 Tokenizar con spaCy o NLTK	9 Registrar y ajustar
5 Lematización y stemming	+ En síntesis, práctica, autocomprobación y glosario

Contenidos del plan y dónde se tratan

Contenido del plan formativo (textual)	Dónde
4. PIPELINE DE PREPROCESAMIENTO Y EVALUACIÓN BÁSICA DE TEXTO	Antes de empezar
CONCEPTOS BÁSICOS DE NLP: TOKEN, PALABRA, LEMA, STOPWORDS Y N-GRAMAS.	Sección 1
TÉCNICAS DE LIMPIEZA: ELIMINACIÓN DE CARACTERES ESPECIALES, HTML Y RUIDO.	Sección 2
NORMALIZACIÓN DE MAYÚSCULAS, MINÚSCULAS Y ESTANDARIZACIÓN DE TEXTO.	Sección 3
TOKENIZACIÓN Y NORMALIZACIÓN CON LIBRERÍAS COMO SPACY O NLTK.	Sección 4
LEMATIZACIÓN Y STEMMING.	Sección 5
VECTORIZACIÓN BÁSICA: COUNTVECTORIZER Y TF-IDF.	Sección 6
ESTRUCTURA MODULAR DE UN PIPELINE DE PREPROCESAMIENTO.	Sección 7
FUNCIONES REUTILIZABLES PARA CADA ETAPA.	Sección 7
MÉTRICAS BÁSICAS DE CALIDAD DE RESPUESTAS: COHERENCIA, RELEVANCIA, BLEU Y ROUGE.	Sección 8

Contenido del plan formativo (textual)	Dónde
REGISTRO DE RESULTADOS DE EVALUACIÓN.	Sección 9
IDENTIFICACIÓN DE AJUSTES INICIALES A PARTIR DE MÉTRICAS OBTENIDAS.	Sección 9

PARA COMENZAR

Antes de empezar

CONTENIDO DEL PLAN 4. PIPELINE DE PREPROCESAMIENTO Y EVALUACIÓN BÁSICA DE TEXTO

En el procesamiento de lenguaje hay una regla que se cumple siempre: **basura entra, basura sale**. Un ticket con etiquetas HTML, una firma larga y un correo personal le cuesta más tokens al modelo, lo distrae y expone datos que no debería recibir. Y un resumen que "se ve bien" no es necesariamente bueno: para decidir entre dos configuraciones hacen falta números.

Este aprendizaje trata de las dos puntas del trabajo con un modelo. Antes, **preparar** el texto: limpiarlo, normalizarlo y dividirlo en unidades con las que se pueda trabajar. Después, **medir** lo que el modelo devuelve con métricas y registrar los resultados para decidir qué ajustar. Todo en Python, con funciones reutilizables que forman un pipeline.

EL PIPELINE DE NUBE SUR



El caso es el resumidor de **Nube Sur**, la empresa ficticia del módulo, con los cinco tickets de la actividad 2 y sus resúmenes de referencia, escritos por el equipo de soporte.

1 Conceptos básicos de NLP

CONTENIDO DEL PLAN CONCEPTOS BÁSICOS DE NLP: TOKEN, PALABRA, LEMA, STOPWORDS Y N-GRAMAS.

El procesamiento de lenguaje natural (NLP, por sus siglas en inglés) tiene un vocabulario propio. Cinco conceptos bastan para este módulo:

- **Token.** Unidad en que se divide un texto para procesarlo. Para una biblioteca como spaCy, un token es casi siempre una palabra o un signo de puntuación. Para un modelo de lenguaje, un token puede ser una palabra completa o un pedazo de ella: "facturación" puede ocupar más de un token.
- **Palabra.** Unidad del idioma separada por espacios o signos. No siempre coincide con un token.

- **Lema.** Forma de diccionario de una palabra: el lema de "pude" y "podemos" es "poder"; el de "tickets" es "ticket".
- **Stopwords.** Palabras muy frecuentes que aportan poco contenido por sí solas, como "el", "de", "que" o "y". Se quitan cuando interesa el tema del texto, no su redacción.
- **N-gramas.** Secuencias de n tokens seguidos. "iniciar" es un unigrama, "iniciar sesión" es un bigrama y "no puedo iniciar" es un trigramas. Capturan expresiones que una palabra sola no dice.

EJEMPLO · NUBE SUR · Una frase del ticket T1

En "No puedo iniciar sesión en la app", los tokens son siete: no, puedo, iniciar, sesión, en, la, app. Sin stopwords quedan las palabras con contenido, como iniciar, sesión y app. El lema de "puedo" es "poder". Y el bigrama "iniciar sesión" identifica el problema mejor que "iniciar" o "sesión" por separado.

2 Limpieza

CONTENIDO DEL PLAN TÉCNICAS DE LIMPIEZA: ELIMINACIÓN DE CARACTERES ESPECIALES, HTML Y RUIDO.

Limpiar es quitar del texto todo lo que no es el mensaje. En los tickets de Nube Sur aparecen tres tipos de basura. El **HTML**: etiquetas como `<p>` o `<br>` y entidades como `´`, que llegan cuando el ticket viene de un formulario web o de un correo. Los **caracteres especiales**, como signos repetidos ("URGENTE!!!"). Y el **ruido**: firmas, frases automáticas como "Enviado desde mi teléfono", enlaces y datos personales como correos.

La herramienta para limpiar es el módulo `re` de Python, con **expresiones regulares**: patrones que describen el texto que se quiere encontrar y reemplazar. Los datos personales no se borran sin más: se reemplazan por una marca, como `[correo]` o `[enlace]`, para que el texto siga teniendo sentido.

```
def limpiar(texto: str) -> str:
    """Quita HTML, firma y datos personales; deja el texto para el modelo."""
    texto = html.unescape(texto)          # &acute; pasa a á
    texto = re.sub(r"<br\s*/?>|<p>", "\n", texto, flags=re.IGNORECASE)
    texto = re.sub(r"<[^>]+>", " ", texto)    # resto de etiquetas
    texto = re.split(r"\n\s*--\s", texto)[0]    # lo que sigue a la firma
    texto = re.sub(r"Enviado desde mi \w+", " ", texto, flags=re.IGNORECASE)
    texto = re.sub(r"\S+@\S+", "[correo]", texto) # datos personales fuera
    texto = re.sub(r"https?://\S+", "[enlace]", texto)
    texto = re.sub(r"!{2,}", "!", texto)
    return re.sub(r"\s+", " ", texto).strip()
```

EJEMPLO · NUBE SUR · El ticket T1, antes y después

Antes: el mensaje de Marta entre etiquetas <p>, con un
 y una firma con su nombre, área y correo. Después de limpiar: "Hola equipo, Desde ayer en la tarde no puedo iniciar sesión en la app de facturación después de cambiar mi contraseña...". Se fueron las etiquetas y la firma con el correo; el problema quedó intacto.

ERROR FRECUENTE · Borrar el problema

El ticket T2 reporta que el PDF muestra "Compañ±Ãa" en vez de "Compañía". Esos caracteres raros **son** el problema, y una limpieza que quita todo lo que no sea letra común los borraría. Antes de limpiar, pregúntate si lo que vas a quitar es ruido o es el mensaje.

3 Normalización

CONTENIDO DEL PLAN NORMALIZACIÓN DE MAYÚSCULAS, MINÚSCULAS Y ESTANDARIZACIÓN DE TEXTO.

Normalizar es hacer que textos equivalentes se escriban igual, para que el programa los trate como iguales. Las operaciones básicas son pasar a **minúsculas** (`texto.lower()`), unificar los **espacios** (varios espacios o saltos de línea pasan a uno) y **estandarizar** el formato: la misma representación de los caracteres con tilde, las mismas comillas, el mismo formato de fechas o de montos.

La normalización depende de para qué se usa el texto. Lo que se envía **al modelo** conviene dejarlo natural, con mayúsculas y tildes, porque el modelo las entiende y las usa. La normalización más agresiva se aplica para **comparar y medir**: minúsculas y, solo entonces, sin tildes, para que "sesión" y "sesion" cuenten como la misma palabra.

```
def quitar_tildes(texto: str) -> str:
    """Pasa 'sesión' a 'sesion'; solo para comparar textos al medir."""
    return "".join(c for c in unicodedata.normalize("NFD", texto)
                    if unicodedata.category(c) != "Mn")
```

EJEMPLO · NUBE SUR · Dos resúmenes que dicen lo mismo

El modelo escribe "Usuario no logra iniciar sesion" (sin tilde) y la referencia dice "El usuario no puede iniciar sesión". Sin normalizar, "sesion" y "sesión" son palabras distintas y la métrica baja injustamente. Con minúsculas y sin tildes solo al medir, cuentan como la misma palabra.

4 Tokenizar con spaCy o NLTK

CONTENIDO DEL PLAN TOKENIZACIÓN Y NORMALIZACIÓN CON LIBRERÍAS COMO SPACY O NLTK.

Separar por espacios no es tokenizar bien: "app," quedaría con la coma pegada y "¿cómo" con el signo. Las bibliotecas de NLP resuelven esto con reglas del idioma. **spaCy** carga un modelo del español, como `es_core_news_sm`, que además de tokenizar entrega, para cada

token, su lema, si es stopword y si es puntuación. **NLTK** ofrece funciones separadas: `word_tokenize` para tokenizar y una lista de stopwords en español; ambas requieren descargar sus recursos la primera vez con `nltk.download`.

```
# Antes, una vez: python -m spacy download es_core_news_sm
nlp = spacy.load("es_core_news_sm")

def tokenizar(texto: str, lematizar: bool = False,
              sin_stopwords: bool = False) -> list[str]:
    """Tokeniza con spaCy; si se pide, lematiza y quita stopwords."""
    tokens = []
    for t in nlp(texto.lower()):
        if t.is_punct or t.is_space or (sin_stopwords and t.is_stop):
            continue
        tokens.append(t.lemma_ if lematizar else t.text)
    return tokens
```

EJEMPLO · NUBE SUR · Los tokens de T1

`tokenizar(limpio, sin_stopwords=True)` sobre el ticket T1 limpio deja, entre otros, `iniciar`, `sesión`, `app`, `facturación` y `contraseña`. Son las palabras que dicen de qué trata el ticket; los artículos, preposiciones y la puntuación quedaron fuera.

ERROR FRECUENTE · Mezclar tokenizadores

Si tokenizas los resúmenes generados con spaCy y las referencias con `split()`, las métricas comparan unidades distintas y los números no significan nada. Todo lo que se compara pasa por la misma función de tokenización.

5 Lematización y stemming

CONTENIDO DEL PLAN LEMATIZACIÓN Y STEMMING.

Tanto la lematización como el stemming reducen las variantes de una palabra a una forma común, para que "facturas", "facturar" y "facturación" puedan contarse juntas. Lo hacen de maneras distintas.

La **lematización** busca el **lema**, la forma de diccionario, usando el análisis gramatical del modelo del idioma: "corriendo" pasa a "correr" y "pude", a "poder". El resultado es siempre una palabra real, pero requiere un modelo como el de spaCy, es más lenta y el modelo pequeño a veces se equivoca. El **stemming** recorta sufijos siguiendo reglas fijas, sin entender la palabra: el `SnowballStemmer("spanish")` de NLTK deja "facturación" y "facturar" en la raíz común "factur". Es rápido y no necesita modelo, pero la raíz no siempre es una palabra.

	Lematización	Stemming
Resultado	Forma de diccionario, palabra real	Raíz recortada, puede no ser palabra

	Lematización	Stemming
Cómo funciona	Análisis gramatical con un modelo	Reglas de recorte de sufijos
Velocidad	Más lenta	Muy rápida
Biblioteca	spaCy (<code>token.lemma_</code>)	NLTK (<code>SnowballStemmer</code>)

EJEMPLO · NUBE SUR · Cuándo usar cada uno en Nube Sur

Para contar los temas más frecuentes de los tickets del mes, el equipo usa **lematización**: el informe muestra palabras reales, como "contraseña" o "factura". Para una búsqueda rápida sobre miles de tickets antiguos, donde la velocidad importa más que la forma, usa **stemming**.

6 Vectorización básica

CONTENIDO DEL PLAN VECTORIZACIÓN BÁSICA: COUNTVECTORIZER Y TF-IDF.

Los algoritmos no comparan palabras: comparan números. **Vectorizar** es representar cada texto como un vector, una lista de números, para poder medir cuánto se parecen dos textos. La biblioteca scikit-learn ofrece dos formas básicas.

CountVectorizer arma un vocabulario con todas las palabras del conjunto de textos y representa cada texto contando cuántas veces aparece cada palabra. Es la "bolsa de palabras": simple, pero trata igual a una palabra rara y a una que aparece en todos los textos. **TF-IDF** corrige eso: multiplica la frecuencia de la palabra en el texto (TF) por un factor que baja cuanto más textos la contienen (IDF). Así, "contraseña" pesa más que "ticket" si casi todos los tickets dicen "ticket".

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vectorizador = TfidfVectorizer(ngram_range=(1, 2)) # palabras y bigramas
X = vectorizador.fit_transform(tickets_limpios)    # una fila por ticket
similitud = cosine_similarity(X[0], X)            # T1 frente a todos
```

EJEMPLO · NUBE SUR · Tickets repetidos

Con TF-IDF sobre los tickets limpios de la semana, el ticket T1 aparece muy cerca de otros dos que también hablan de "iniciar sesión" y "contraseña". El equipo los agrupa y los atiende juntos. Los embeddings del aprendizaje esperado 2 hacen lo mismo con más precisión, porque captan el significado y no solo las palabras compartidas.

ERROR FRECUENTE · Stopwords en español con scikit-learn

El parámetro `stop_words` de los vectorizadores de scikit-learn solo trae incorporada la lista en inglés. Si escribes `stop_words="english"` sobre textos en español, no quita nada útil. Entrega una lista en español, por ejemplo la de NLTK.

7 Un pipeline modular

CONTENIDO DEL PLAN ESTRUCTURA MODULAR DE UN PIPELINE DE PREPROCESAMIENTO. · FUNCIONES REUTILIZABLES PARA CADA ETAPA.

Un **pipeline** de preprocesamiento es la secuencia de etapas por la que pasa cada texto. Su **estructura modular** consiste en que cada etapa sea una **función reutilizable** con una sola responsabilidad: `limpiar` solo limpia, `tokenizar` solo tokeniza, `ngramas` solo arma n-gramas, `medir` solo calcula métricas. Las funciones se encadenan, y cada una se puede probar, cambiar y reutilizar por separado.

La modularidad tiene tres ventajas concretas. Se **prueba** cada etapa con casos pequeños, como un `assert` que confirma que `limpiar` quita `<p>`. Se **reutiliza** el mismo pipeline para los tickets, para los resúmenes generados y para las referencias, lo que garantiza que se comparen igual. Y se **cambia** una etapa, por ejemplo agregar lematización, activando un parámetro y no reescribiendo todo.

```
def ngramas(tokens: list[str], n: int = 2) -> list[tuple[str, ...]]:
    """Devuelve los n-gramas consecutivos de una lista de tokens."""
    return list(zip(*(tokens[i:] for i in range(n))))

limpio = limpiar(ticket)
tokens = tokenizar(limpio, sin_stopwords=True)
bigramas = ngramas(tokens, 2)
```

EJEMPLO · NUBE SUR · Las funciones de Nube Sur

El archivo `preprocesar.py` reúne `limpiar`, `quitar_tildes`, `tokenizar` y `ngramas`; el archivo `evaluar.py` importa `quitar_tildes` y agrega `medir` y `registrar`. El notebook de la actividad 2 solo llama a esas funciones, así que el mismo código sirve después en el servidor del resumidor.

8 Métricas de calidad

CONTENIDO DEL PLAN MÉTRICAS BÁSICAS DE CALIDAD DE RESPUESTAS: COHERENCIA, RELEVANCIA, BLEU Y ROUGE.

Para evaluar un resumen generado se compara con un **resumen de referencia**, escrito por una persona. Hay dos métricas automáticas clásicas. **ROUGE** mide cuánto de la referencia aparece en el resumen generado. ROUGE-1 compara palabras sueltas, y ROUGE-L, la secuencia común más larga, que respeta el orden. Se suele reportar su F1, que combina precisión y cobertura. **BLEU** mide cuánto del resumen generado aparece en la referencia, contando palabras y secuencias de hasta cuatro, y penaliza los resúmenes demasiado cortos. Las dos van de 0 a 1.

Las métricas automáticas miden coincidencia de palabras, no verdad. Un resumen que dice "funciona" donde la referencia dice "no funciona" puede tener un ROUGE altísimo. Por eso se complementan con dos métricas humanas, en una escala de 1 a 3. La **coherencia** vale 3 si el resumen es una oración gramatical y sin contradicciones. La **relevancia** vale 3 si nombra la falla y el producto del ticket sin agregar nada que no esté en él.

EJEMPLO · NUBE SUR · Un cálculo para comprobar

Referencia de T1: "El usuario no puede iniciar sesión en la app de facturación tras cambiar su contraseña." Generado: "Usuario no logra iniciar sesión en la app de facturación después de cambiar la contraseña." Ambos tienen 15 palabras y comparten 11, en el mismo orden:

ROUGE-1 = ROUGE-L = $11/15 = 0,733$. Si con este par obtienes un valor muy distinto, tu tokenizador está perdiendo las palabras con tilde.

ERROR FRECUENTE · El tokenizador por defecto

La biblioteca `rouge-score` usa por defecto un tokenizador pensado para inglés, que descarta las letras con tilde y la ñ: "sesión" y "contraseña" se parten y las métricas en español salen mal. Entrégale un tokenizador que conserve esas letras, como el `TokenizadorES` de `evaluar.py`.

9 Registrar y ajustar

CONTENIDO DEL PLAN REGISTRO DE RESULTADOS DE EVALUACIÓN · IDENTIFICACIÓN DE AJUSTES INICIALES A PARTIR DE MÉTRICAS OBTENIDAS.

Una métrica calculada y no registrada no sirve para decidir. El **registro de resultados** sigue un **protocolo**: los mismos tickets, las mismas referencias, el mismo tokenizador y una fila por prueba en un archivo CSV. En Nube Sur cada fila tiene el ticket, la estrategia y la versión del prompt, la temperature, ROUGE-1, ROUGE-L, BLEU, los tokens usados, la coherencia, la relevancia y un comentario. Se registran también las pruebas que salen mal: son las que más enseñan.

Con el registro completo se **identifican los ajustes**. Primero se comparan los promedios por configuración, para ver qué estrategia y qué temperature rinden mejor. Después se miran los peores casos, que suelen apuntar a un problema concreto del prompt. Y finalmente se decide qué cambiar, uno a la vez, y se vuelve a medir.

Lo que muestran los números	Ajuste inicial
Una configuración con temperature alta baja en relevancia	Bajar temperature para tareas de extracción
Un solo ticket con métricas bajas en todas las versiones	Revisar ese caso: formato especial, limpieza o un ejemplo que falta
ROUGE alto pero relevancia baja	Revisar el contenido: puede haber datos agregados o invertidos

Lo que muestran los números	Ajuste inicial
Tokens altos sin mejora en las métricas	Acortar el prompt o quitar ejemplos que no aportan

EJEMPLO · NUBE SUR · La recomendación del laboratorio

En la actividad 2, el registro muestra que el few-shot v3 con temperature 0,2 obtiene el ROUGE-L promedio más alto y coherencia y relevancia 3 en los cinco tickets. El zero-shot queda cerca en ROUGE, pero falla el formato del ticket T4, que es una pregunta. Con temperature 0,9 baja la relevancia porque aparece información que no estaba en el ticket. La recomendación se apoya en esa tabla, no en la impresión de quién lee.

En síntesis

- Token, palabra, lema, stopwords y n-gramas son el vocabulario básico del NLP.
- Limpiar quita HTML, caracteres especiales, ruido y datos personales, sin borrar el problema que el texto reporta.
- Normalizar unifica mayúsculas, espacios y formato; sin tildes solo para comparar y medir.
- spaCy y NLTK tokenizan con reglas del idioma; todo lo que se compara pasa por el mismo tokenizador.
- La lematización da palabras reales con un modelo; el stemming recorta raíces por reglas, más rápido.
- CountVectorizer cuenta palabras y TF-IDF pondera por lo distintivas que son; los dos permiten medir similitud.
- Un pipeline modular encadena funciones reutilizables, una por etapa, que se prueban por separado.
- ROUGE y BLEU miden coincidencia con una referencia; coherencia y relevancia, con pauta humana, miden sentido.
- Los resultados se registran con un protocolo en un CSV y los ajustes se deciden con esos números.

SIGUIENTE PASO

Para practicar

- **Notebook guiado "Tu primera llamada a un modelo", secciones 7 y 8.** Limpias el ticket T1 y lo tokenizas con spaCy, con celdas que comprueban el resultado.
- **Actividad 2, "Laboratorio de prompts y métricas", pasos 1, 4 y 5.** Programas el preprocesamiento, mides cada resumen con ROUGE, BLEU, coherencia y relevancia, registras todo en `resultados.csv` y recomiendas una configuración con números.
- **Documentación oficial.** La de spaCy (spacy.io), NLTK (nltk.org) y scikit-learn (scikit-learn.org), con ejemplos de cada función.

Autocomprobación

Responde con tus palabras antes de mirar las respuestas. Si dudas en alguna, vuelve a la sección que se indica.

- 1 ¿Qué diferencia hay entre el lema y el stem de una palabra? Da un ejemplo de cada uno.

- 2 Un resumen generado obtiene ROUGE-1 de 0,85, pero dice que el botón "funciona" cuando el ticket dice que "no funciona". ¿Qué métrica lo detecta y por qué ROUGE no?

- 3 Calculas ROUGE para resúmenes en español y todos los valores salen más bajos de lo esperado. ¿Qué revisas primero?

RESPUESTAS

1. El lema es la forma de diccionario, que se obtiene con análisis gramatical, por ejemplo "poder" para "pude". El stem es la raíz que queda al recortar sufijos por reglas y puede no ser una palabra, como "factur" para "facturación" (sección 5).
2. La relevancia, valorada por una persona con la pauta de 1 a 3. ROUGE solo mide cuántas palabras coinciden con la referencia, no si el sentido es correcto (sección 8).
3. El tokenizador. El de `rouge-score` por defecto descarta las letras con tilde y la ñ, así que las palabras se parten y dejan de coincidir. Hay que usar un tokenizador que las conserve y normalizar igual la referencia y el resumen (secciones 3 y 8).

Glosario

BLEU	Métrica que mide cuánto de un texto generado aparece en la referencia, con penalización por ser demasiado corto.
Lema	Forma de diccionario de una palabra, como "poder" para "pude".
N-grama	Secuencia de n tokens consecutivos, como el bigrama "iniciar sesión".
ROUGE	Métrica que mide cuánto de un texto de referencia aparece en un texto generado.
Stemming	Técnica que reduce una palabra a su raíz recortando sufijos con reglas fijas.
Stopwords	Palabras muy frecuentes y de poco contenido, como artículos y preposiciones, que se quitan al analizar temas.
TF-IDF	Forma de vectorizar un texto que pondera cada palabra por su frecuencia en el texto y su rareza en el conjunto.
Token	Unidad en que se divide un texto para procesarlo; puede ser una palabra, parte de una palabra o un signo.